

1. Classifying each public method of RatNum:
 - a. **RatNum(int n)** is a **creator** because it creates new objects of the type
 - b. **RatNum(int n, int d)** is a **creator** because it creates new object of the type
 - c. **isNaN()** is an **observer** because it takes objects of the abstract type and returns objects of a different type
 - d. **isNegative()** is an **observer** because it takes objects of the abstract type and returns objects of a different type
 - e. **isPositive()** is an **observer** because it takes objects of the abstract type and returns objects of a different type
 - f. **compareTo(RatNum rn)** is a **producer** because it creates new objects from old objects
 - g. **doubleValue()** is a **producer** because it creates new objects from old objects
 - h. **intValue()** is a **producer** because it creates new objects from old objects
 - i. **floatValue()** is a **producer** because it creates new objects from old objects
 - j. **longValue()** is a **producer** because it creates new objects from old objects
 - k. **negate()** is a **producer** because it creates new objects from old objects
 - l. **add(RatNum arg)** is a **producer** because it creates new objects from old objects
 - m. **sub(RatNum arg)** is a **producer** because it creates new objects from old objects
 - n. **mult(RatNum arg)** is a **producer** because it creates new objects from old objects
 - o. **div(RatNum arg)** is a **producer** because it creates new objects from old objects
 - p. **hashCode()** is a **producer** because it creates new objects from old objects
 - q. **equals(Object obj)** is an **observer** because it takes objects of the abstract type and returns objects of a different type
 - r. **toString()** is a **producer** because it creates new objects from old objects
 - s. **valueOf(String ratStr)** is a **creator** because it creates new objects of the type
2. In Java, it is common practice to assume that instance methods in a class will be called on valid instances of objects, resulting in checks for “this != null” being left out of our methods. The check for “arg != null” being included is because someone could within reason refer to an arg that is null, resulting in a NullPointerException. The check ensures that this exception does not occur.
3. RatNum.valueOf(String) is declared as a class method (has static modifier) because its functionality does not depend on an instance of the RatNum class. An alternative way to implement RatNum.valueOf(String) would be to create a separate class which creates instances of RatNum from string inputs. However, this is much less efficient than the current implementation.
4. The RatNum class is designed to be immutable, so any instance of the class cannot be modified, and since we are directly assigning new values to a RatNum instance, this would fail. Next, the “@requires” clause would be broken, as we would no longer be able to verify if the conditions were valid due to these changes. Finally, there is no “@modifies” clause, which would indicate what in the function is being modified.

5. The `checkRep()` method only needs to be called at the end of constructors because of the immutability of instances of `RatNum`. Additionally, the variables being assigned values in the current constructors are declared with the keyword “final”, which also prevents them from being changed after initial assignment. It is the job of a constructor to adhere to its invariants at the beginning of its calling as well, so in the end it is only necessary to call `checkRep()` at the end of each constructor to ensure their invariants still hold.